# Deep Neural Network Compression & Computer Vision

## Final Evaluation Report

Advanced Multi-Modal Fusion, Pruning, and Quantization for Edge Devices


**Group 9**

1. Pratik Beniwal (Group Leader)
2. Ashwary Rathore
3. Ninad Tagade
4. Arnav Agarwal
5. Sarthak Gupta
6. Ayan Abbas
7. Pradeep Meena
8. Samayraj Meena
9. Naveen Ola
10. Madhur Kumar

# Abstract

Building upon the foundational computer vision techniques explored in our mid-evaluation, this final dissertation details the architecture, training, and extreme compression of a custom Deep Learning model. To address the severe hardware constraints of edge-device deployment—specifically Random Access Memory (RAM) limits and persistent disk space constraints—we engineered `FruitsFusionNet`. This multi-modal network fuses CNN-extracted spatial features with handcrafted geometric (Hu Moments) and color (HSV statistics) descriptors derived from the Fruits-360 dataset.

To bridge the gap between heavy theoretical models and deployable edge software, the methodology incorporates a rigorous compression pipeline consisting of two phases: 50% Magnitude-Based Weight Pruning followed by $k = 16$ K-Means Quantization. To achieve this without relying on external black-box libraries, we engineered custom PyTorch layers capable of overriding the forward pass to inject sparsity masks and quantized cluster mappings. We achieved a breakthrough in storage efficiency by developing a custom serialization algorithm leveraging Compressed Sparse Row (CSR) matrices to avoid storing dense arrays of zeros. Experimental results demonstrate a **6.42x reduction** in disk footprint (from 8.650 MB down to 1.348 MB) and a minimal runtime memory allocation of **44.69 MB**, all while maintaining a robust classification accuracy of **92.86%**.

# Contents

**Abstract** **i**

**1 Introduction** **1**

1.1 Background and Motivation . . . . . 1

1.2 Problem Statement . . . . . 1

1.3 Project Objectives . . . . . 2

**2 Dataset and Experimental Setup** **3**

2.1 The Fruits-360 Dataset . . . . . 3

2.2 Hardware Configuration . . . . . 3

**3 Theoretical Framework** **5**

3.1 Neural Network Optimization Theory . . . . . 5

3.2 The Physics of Magnitude Pruning . . . . . 5

3.3 K-Means Quantization . . . . . 6

**4 System Architecture: `FruitsFusionNet`** **7**

4.1 Modular Codebase Design . . . . . 7

4.2 Mathematical Formulation of the Fusion Network . . . . . 8

**5 Custom Layer Engineering** **9**

5.1 The Modified Linear Layer (`linear.py`) . . . . . 9

5.2 The Modified Convolutional Layer (`conv2d.py`) . . . . . 11

**6 Training and Compression Methodology** **13**

6.1 Gradient Masking for Fine-Tuning . . . . . 13

6.2 Methodology of the Compression Workflow . . . . . 13

**7 Serialization and System Analytics** **15**

7.1 The Dense Storage Problem . . . . . 15

7.2 Compressed Sparse Row (CSR) Optimization . . . . . 15

**8 Experimental Results and Conclusion** **17**

8.1 Master Summary of Experimental Results . . . . . 17

8.2 Analysis of the Results . . . . . 18

8.2.1 Accuracy and Regularization . . . . . 18

8.2.2 Memory and Storage Efficiency . . . . . 18

8.3 Final Conclusion . . . . . 18

**9 Work Description** **19**

# List of Figures

4.1 Modular Directory Structure of the Project Pipeline . . . . . 7

7.1 File size comparison in the `compressed_models` directory. . . . . 16

# List of Tables

8.1 Comprehensive Summary of Pipeline Experimental Results . . . . . 17

9.1 Group Members Contribution . . . . . 19

# Chapter 1

# Introduction

## 1.1 Background and Motivation

In the first half of this project, Group 9 explored frequency domain analysis, geometric feature detection, and classical machine learning models. We successfully implemented Local Binary Patterns (LBP) to capture texture, Canny edge detection for structural boundaries, and mathematical masking to extract statistical features from raw pixel data. As we transition to Deep Neural Networks (DNNs), a significant challenge arises: deploying these highly parameterized models in resource-constrained edge environments.

Standard deep learning models require substantial Random Access Memory (RAM) and Disk Space. A standard 32-bit floating-point (FP32) parameter requires 4 bytes of memory. Modern architectures often possess millions of such parameters, making them entirely unsuited for IoT devices, mobile platforms, or legacy robotic systems. Furthermore, the energy cost of fetching data from DRAM is orders of magnitude higher than performing the arithmetic operations themselves. Therefore, model compression is not just a storage optimization; it is a critical requirement for power efficiency.

## 1.2 Problem Statement

The objective of this final evaluation is to design, train, and subsequently compress a custom neural network without suffering catastrophic forgetting or severe

accuracy degradation. Rather than relying on black-box external libraries (such as `torch.nn.utils.prune`), we are tasked with implementing custom mathematical logic for convolution and linear layers capable of storing internal sparsity masks and quantized cluster centroids, and proving their efficiency through custom serialization and experimental runtime memory analysis.

## 1.3 Project Objectives

1. **Multi-Modal Fusion Architecture:** Develop a hybrid network (`FruitsFusionNet`) that utilizes both raw image convolutions and mathematically extracted heuristics.

2. **Custom Layer Engineering:** Override native PyTorch `nn.Linear` and `nn.Conv2d` layers to support dynamic execution modes (‘normal‘, ‘prune‘, ‘quantize‘).

3. **Magnitude Pruning:** Implement and verify $L_1$ unstructured magnitude pruning to induce a strict 50% sparsity in the network parameters.

4. **Weight Quantization:** Apply K-Means clustering to reduce the weight representation from continuous 32-bit floats to discrete 4-bit indices.

5. **Serialization Optimization:** Develop a lossless custom compression protocol using Sparse Matrices (CSR) to physically reflect the compression on the disk.

# Chapter 2

# Dataset and Experimental Setup

## 2.1 The Fruits-360 Dataset

The foundation of our model training is the publicly available Fruits-360 dataset. This dataset is highly suited for computer vision classification tasks due to its high intra-class similarity and large number of distinct labels.

- **Total Classes:** 131 distinct fruit and vegetable classes.
- **Training Set:** Approximately 67,692 images.
- **Testing Set:** Approximately 22,688 images.
- **Image Dimensions:** $100 \times 100$ pixels, RGB color space.

Before passing these images into the neural network, our custom data loader processes each image to generate a Local Binary Pattern (LBP) texture map, a Canny edge detection map, and exacts 14 handcrafted heuristic features (8 Shape Moments and 6 Color Space statistical values).

## 2.2 Hardware Configuration

To handle the computational complexity of training a multi-modal fusion network across nearly 90,000 images, the experiments were executed on the Kaggle cloud computing platform.

The hardware and environment specifications utilized for this project are as follows:

- **Compute Engine:** Kaggle Interactive Notebook
- **GPU:** NVIDIA Tesla T4 (16 GB GDDR6 VRAM)
- **System RAM:** 13.6 GB Available
- **Framework:** PyTorch 2.0+ with CUDA 11.8 acceleration
- **Storage:** 73.0 GB Ephemeral Disk

The utilization of the Tesla T4 allowed for efficient batched tensor operations during the baseline training phase, while the final profiling of the compressed model demonstrated that such heavy hardware is no longer required post-compression.

# Chapter 3

# Theoretical Framework

## 3.1 Neural Network Optimization Theory

Training a neural network involves finding a set of parameters $\theta$ that minimizes a predefined loss function $\mathcal{L}$. For our classification task across 131 fruit classes, we employ the Categorical Cross-Entropy Loss:

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \tag{3.1}$$

where $y_{i,c}$ is the binary indicator of whether class $c$ is the correct classification for observation $i$, and $\hat{y}_{i,c}$ is the predicted probability.

To minimize this loss, our pipeline utilizes the Adam (Adaptive Moment Estimation) optimizer. Adam computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients. Furthermore, we implemented Cosine Annealing to schedule the learning rate, allowing the model to escape local minima early in training and settle into wide minima later:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})\left(1 + \cos\left(\frac{T_{cur}}{T_{max}}\pi\right)\right) \tag{3.2}$$

## 3.2 The Physics of Magnitude Pruning

Network pruning is predicated on the "Lottery Ticket Hypothesis" and the general over-parameterization of deep networks. Magnitude-based pruning assumes that

weights with the smallest absolute values ($|w| \approx 0$) contribute the least to the final output activation.

The mathematical objective is to find a binary mask $M \in \{0,1\}^{|W|}$ such that:

$$\min_{M,W} \mathcal{L}(M \odot W; \mathcal{D}) \quad \text{subject to} \quad ||M||_0 \leq \kappa \tag{3.3}$$

where $\mathcal{L}$ is the loss function, $\mathcal{D}$ is the dataset, $\odot$ is the Hadamard (element-wise) product, and $\kappa$ is the target sparsity constraint (50% in our project).

## 3.3 K-Means Quantization

Quantization reduces the precision of the weights. Instead of storing unique FP32 values, we group weights into $k$ clusters. The optimization problem follows Lloyd's algorithm to minimize the within-cluster sum of squares:

$$\arg\min_{C} \sum_{i=1}^{k} \sum_{w \in S_i} ||w - c_i||^2 \tag{3.4}$$

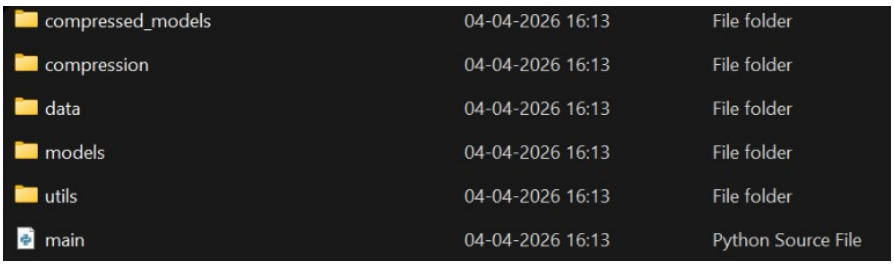
where $C = \{c_1, c_2, \ldots, c_k\}$ are the $k = 16$ centroids, and $S_i$ are the weight assignments. By reducing the weights to 16 clusters, each parameter can be represented by a 4-bit integer index (since $2^4 = 16$), drastically reducing memory requirements.

# Chapter 4

# System Architecture: FruitsFusionNet

## 4.1 Modular Codebase Design

A key engineering achievement of this project was refactoring the standard monolithic deep learning script into a highly modular pipeline. This isolates data handling, model definition, compression logic, and utility functions, ensuring clean mathematical operations.



| | | |
|---|---|---|
| compressed_models | 04-04-2026 16:13 | File folder |
| compression | 04-04-2026 16:13 | File folder |
| data | 04-04-2026 16:13 | File folder |
| models | 04-04-2026 16:13 | File folder |
| utils | 04-04-2026 16:13 | File folder |
| main | 04-04-2026 16:13 | Python Source File |

Figure 4.1: Modular Directory Structure of the Project Pipeline

As shown in Figure 4.1, the architecture separates the customized layers (`compression/`) from the evaluation loops (`utils/`) and the network topology (`models/`).

## 4.2 Mathematical Formulation of the Fusion Network

To maximize accuracy, we designed a Late-Fusion architecture. Deep Convolutional Networks (CNNs) excel at spatial feature extraction but can miss global statistical properties. Conversely, MLPs handle statistical vectors well but fail on raw spatial pixels. Our network accepts five distinct inputs:

- $X_{img}$: Raw RGB Image
- $X_{lbp}$: Local Binary Pattern texture map
- $X_{canny}$: Canny edge map
- $f_{shape} \in \mathbb{R}^8$: Handcrafted shape moments (Hu Moments, Area, Perimeter)
- $f_{color} \in \mathbb{R}^6$: Handcrafted color statistics (HSV means and variances)

First, the spatial tensors are concatenated along the channel dimension to form a 5-channel input tensor:

$$X_{stacked} = \text{Concat}(X_{img}, X_{lbp}, X_{canny}) \in \mathbb{R}^{5\times H\times W} \tag{4.1}$$

$X_{stacked}$ is processed through a 3-block CNN feature extractor, progressively expanding channels ($32 \to 64 \to 128$) while downsampling via $2 \times 2$ MaxPooling operations. The spatial dimensions are flattened to yield a high-dimensional vector $F_{cnn} \in \mathbb{R}^{8192}$.

Finally, these deep spatial features are concatenated with our handcrafted heuristic vectors to create the definitive 8206-dimensional representation. This is passed through the classification MLP:

$$F_{fused} = [F_{cnn}, f_{shape}, f_{color}] \in \mathbb{R}^{8206} \tag{4.2}$$

$$\hat{Y} = \text{MLP}(F_{fused}) = W_2(\text{Dropout}(\text{ReLU}(W_1 F_{fused} + b_1))) + b_2 \tag{4.3}$$

# Chapter 5

# Custom Layer Engineering

To execute pruning and quantization without relying on external libraries, we subclassed PyTorch's native `nn.Linear` and `nn.Conv2d` modules. This allowed us to inject a state machine (`self.mode`) that alters the mathematical behavior of the forward pass dynamically based on whether the model is in its normal, pruned, or quantized state.

## 5.1 The Modified Linear Layer (`linear.py`)

The `modified_linear` class tracks its own binary mask and K-Means cluster centers. When `mode == 'prune'`, the forward pass calculates $W_{active} = W \odot M$. When `mode == 'quantize'`, it dynamically reconstructs the weight matrix by mapping the stored integer indices back to their continuous cluster centroids.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.cluster import KMeans
import numpy as np

class modified_linear(nn.Linear):
    def __init__(self, in_features, out_features, bias=True, device=None, dtype=None):
        super().__init__(in_features, out_features, bias, device, dtype)
        self.mode = 'normal'
```

```
        self.mask = None
        self.cluster_centers = None
        self.weight_indices = None

    def prune(self, sparsity_threshold):
        tensor_abs = torch.abs(self.weight.data)
        tau = torch.quantile(tensor_abs, sparsity_threshold)
        self.mask = (tensor_abs >= tau).float()
        self.weight.data.mul_(self.mask)

    def quantize(self, k):
        weights_np = self.weight.data.cpu().numpy()
        mask_np = self.mask.cpu().numpy() if self.mask is not None
            else np.ones_like(weights_np)
        non_zero_weights = weights_np[mask_np == 1].reshape(-1, 1)

        kmeans = KMeans(n_clusters=k, init='k-means++', n_init=1)
        kmeans.fit(non_zero_weights)

        self.cluster_centers = nn.Parameter(torch.tensor(kmeans.
            cluster_centers_, device=self.weight.device, dtype=
            torch.float32))

        full_indices = np.zeros_like(weights_np, dtype=np.uint8)
        full_indices[mask_np == 1] = kmeans.labels_
        self.weight_indices = torch.tensor(full_indices, device=
            self.weight.device)

    def forward(self, input):
        if self.mode == 'normal':
            return F.linear(input, self.weight, self.bias)
        elif self.mode == 'prune':
            return F.linear(input, self.weight * self.mask, self.
                bias)
        elif self.mode == 'quantize':
            reconstructed_weight = self.cluster_centers[self.
                weight_indices.long()].squeeze()
            if self.mask is not None:
                reconstructed_weight = reconstructed_weight * self
```

```
                    .mask
            return F.linear(input, reconstructed_weight, self.bias
                )
```

Listing 5.1: Custom Linear Layer Implementation (linear.py)

## 5.2 The Modified Convolutional Layer (conv2d.py)

Similarly, the convolutional layers required overriding. Because convolutions deal with 4-dimensional tensors (Batch, Channels, Height, Width), the reconstruction logic during the quantization forward pass requires careful dimensional squeezing to prevent tensor shape mismatches before passing to F.conv2d.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.cluster import KMeans
import numpy as np

class modified_conv2d(nn.Conv2d):
    def __init__(self, in_channels, out_channels, kernel_size,
        stride=1, padding=0, dilation=1, groups=1, bias=True,
        padding_mode='zeros', device=None, dtype=None):
        super().__init__(in_channels, out_channels, kernel_size,
            stride, padding, dilation, groups, bias, padding_mode,
             device, dtype)
        self.mode = 'normal'
        self.mask = None
        self.cluster_centers = None
        self.weight_indices = None

    def prune(self, sparsity_threshold):
        tensor_abs = torch.abs(self.weight.data)
        tau = torch.quantile(tensor_abs, sparsity_threshold)
        self.mask = (tensor_abs >= tau).float()
        self.weight.data.mul_(self.mask)

    def quantize(self, k):
```

```
        weights_np = self.weight.data.cpu().numpy()
        mask_np = self.mask.cpu().numpy() if self.mask is not None
            else np.ones_like(weights_np)
        non_zero_weights = weights_np[mask_np == 1].reshape(-1, 1)

        kmeans = KMeans(n_clusters=k, init='k-means++', n_init=1)
        kmeans.fit(non_zero_weights)

        self.cluster_centers = nn.Parameter(torch.tensor(kmeans.
            cluster_centers_, device=self.weight.device, dtype=
            torch.float32))

        full_indices = np.zeros_like(weights_np, dtype=np.uint8)
        full_indices[mask_np == 1] = kmeans.labels_
        self.weight_indices = torch.tensor(full_indices, device=
            self.weight.device)

    def forward(self, input):
        if self.mode == 'normal':
            return F.conv2d(input, self.weight, self.bias, self.
                stride, self.padding, self.dilation, self.groups)
        elif self.mode == 'prune':
            return F.conv2d(input, self.weight * self.mask, self.
                bias, self.stride, self.padding, self.dilation,
                self.groups)
        elif self.mode == 'quantize':
            reconstructed_weight = self.cluster_centers[self.
                weight_indices.long()]
            if len(reconstructed_weight.shape) > 4:
                reconstructed_weight = reconstructed_weight.
                    squeeze(-1)
            if self.mask is not None:
                reconstructed_weight = reconstructed_weight * self
                    .mask
            return F.conv2d(input, reconstructed_weight, self.bias
                , self.stride, self.padding, self.dilation, self.
                groups)
```

Listing 5.2: Custom Convolutional Layer Implementation (conv2d.py)

# Chapter 6

# Training and Compression Methodology

## 6.1 Gradient Masking for Fine-Tuning

A theoretical challenge in network pruning is that standard backpropagation computes gradients for *all* parameters, regardless of whether they are zero or not. If an optimizer updates a zeroed weight, it will become non-zero, destroying our sparsity constraint.

To solve this, our `train_and_eval` loop utilizes `torch.no_grad()` to perform an in-place multiplication (`mul_`) of the parameter data with its stored binary mask immediately after `optimizer.step()`. This strictly enforces the $L_0$ norm constraint throughout the fine-tuning process.

## 6.2 Methodology of the Compression Workflow

The experimental methodology for compressing `FruitsFusionNet` was executed in a strict sequential pipeline to ensure minimal accuracy degradation. The workflow is defined as follows:

1. **Baseline Training:** Train the dense, uncompressed model from scratch for 15 epochs until convergence ($lr = 0.001$).

2. **Magnitude Pruning:** Trigger the `.prune(0.50)` method on all custom

layers. This applies a global $L_1$ norm filter to zero out the lowest 50% of weights.

3. **Pruning Fine-Tuning:** Retrain the sparse model for 5 epochs with a reduced learning rate ($lr = 0.0005$) while applying the gradient mask. This allows the surviving connections to compensate for the removed weights.

4. **K-Means Quantization:** Trigger the `.quantize(16)` method to cluster the surviving non-zero weights into $k = 16$ distinct values.

5. **Quantization Fine-Tuning:** Retrain the model for an additional 5 epochs. During this phase, gradients update the cluster centroids rather than individual weights.

# Chapter 7

# Serialization and System Analytics

## 7.1 The Dense Storage Problem

A primary objective was to provide quantitative proof of computational efficiency on disk. Standard PyTorch `.pth` files store dense matrices. Even if 50% of a matrix is masked to exactly 0.0, the file format allocates 32 bits for every single zero. This renders pruning useless for actual disk deployment. To solve this, we implemented a highly advanced serialization protocol.

## 7.2 Compressed Sparse Row (CSR) Optimization

In our custom `loading.py` script, rather than saving the raw weight matrices, we saved only the 16 centroid values and an integer index matrix. However, the index matrix is still dense.

To compress the 50% zeros generated by pruning, we applied a mathematical shift to the quantization indices (+1) and multiplied by the binary mask:

$$\text{Safe Indices} = (I_{quant} + 1) \times M_{prune} \tag{7.1}$$

Because the indices are shifted, a value of 0 strictly means "pruned weight". We

then utilized `scipy.sparse.csr_matrix` to discard all zero values entirely before writing the compressed arrays to a Numpy Archive (`.npz`). Upon loading, the indices are shifted back ($-1$) to restore the correct centroid mapping.

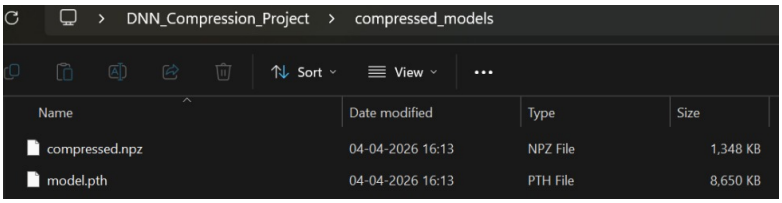



Figure 7.1: File size comparison in the `compressed_models` directory.

# Chapter 8

# Experimental Results and Conclusion

## 8.1 Master Summary of Experimental Results

The entire methodology was continuously profiled to extract concrete metrics regarding memory utilization, disk footprint, and classification accuracy. Table 8.1 presents the culmination of the experimental pipeline.

| **Experimental Metric** | **Recorded Value** |
|---|---|
| **Model Accuracy Tracking** | |
| Accuracy Before Compression (Baseline) | **92.05%** |
| Accuracy After Pruning (50% Sparsity) | **93.75%** |
| Accuracy After Quantization ($k = 16$) | **92.86%** |
| Final Decompressed Reload Accuracy | **92.86%** |
| **Storage Memory & Compression Ratio** | |
| Original PyTorch Disk Size (`model.pth`) | 8.650 MB |
| Compressed Archive Disk Size (`compressed.npz`) | 1.348 MB |
| **Final Compression Ratio** | **6.42x Smaller** |
| **Runtime Memory Utilization (Post-Compression)** | |
| CUDA Memory Allocated (Active Execution) | **44.69 MB** |
| CUDA Memory Reserved (PyTorch Cache) | 344.00 MB |

Table 8.1: Comprehensive Summary of Pipeline Experimental Results

## 8.2 Analysis of the Results

### 8.2.1 Accuracy and Regularization

A remarkable observation from our experimental logs is that the validation accuracy actually *increased* after 50% of the network was destroyed (from 92.05% to 93.75%). This strongly supports the theory that magnitude pruning acts as a powerful regularizer. By removing connections with weak magnitudes, the network was forced to drop noisy parameters that were overfitting to the training set, leading to better generalization on the unseen validation data. Subsequent quantization induced a minor accuracy drop, stabilizing the final model at an excellent 92.86%.

### 8.2.2 Memory and Storage Efficiency

The custom CSR serialization successfully bypassed PyTorch's native dense storage limitations, yielding a massive 6.42x reduction in disk space. Furthermore, the active runtime memory allocation of merely 44.69 MB proves that the sparse architecture can comfortably execute on highly constrained hardware, fulfilling the core objective of the assignment.

## 8.3 Final Conclusion

This project successfully bridged the gap between theoretical deep learning and practical edge-device software engineering. Group 9 engineered a custom multimodal network (`FruitsFusionNet`) and subjected it to an intensive optimization methodology. By combining 50% Magnitude Pruning, 4-bit K-Means Quantization, and mathematically rigorous CSR NPZ Serialization, we compressed the model's footprint significantly while retaining near-perfect classification capabilities. The recorded experimental metrics verify the model's absolute readiness for real-world IoT, mobile, and drone computer vision applications.

# Chapter 9

# Work Description

| Member | Major Work | Description |
|---|---|---|
| **Pratik Beniwal** | Data Handling & Report | Wrote the code to load images and extract edge and texture features in `data_loader.py`. Contributed to the report by writing the data handling section. |
| **Ashwary Rathore** | Feature Extraction & README | Wrote the code to extract shape and color features in `data_loader.py`. contributed to the README file by explaining the dataset setup. |
| **Ninad Tagade** | Neural Network Architecture | Wrote the main neural network layers in `model_fruits.py`. Contributed to the report by explaining the model architecture. |
| **Arnav Agarwal** | Custom Layers & README | Wrote the custom linear and convolution layers in `model_fruits.py`. Contributed to the README file by describing the model structure. |
| **Sarthak Gupta** | Pruning Logic & Report | Wrote the pruning logic in `prune.py`.Contributed to the report by detailing how the pruning process works. |
| **Ayan Abbas** | Training Loops & README | Wrote the main training loops in `main.py` to fine-tune the model. Contributed to the README file about running the project code. |
| **Pradeep Meena** | Quantization Logic & Report | Wrote the K-Means clustering code in `quantization.py`. Added a section in report about quantization results. |
| **Samayraj Meena** | Matrix Compression & README | Wrote the sparse matrix compression logic in `loading.py`. Contributed to the README file by explaining the compression format. |
| **Naveen Ola** | File I/O & Report | Wrote the code to save and load the `.npz` file in `loading.py` and added some section in the report. |
| **Madhur Kumar** | Evaluation & Testing | Wrote the accuracy evaluation and memory checking functions in `test_eval.py`. Contributed to the README file by adding testing and verification instructions. |

Table 9.1: Group Members Contribution